

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME:K.VIJAYALAKSHMI

REG.NO:192511189

Question 1

Identify and fix the error in the SQL JOIN query.

Incorrect Query

*SELECT **

FROM Employee, Department

WHERE Employee.DeptID = Department.DeptID;

Correct Query

SELECT E.EmpID,

E.EmpName,

D.DeptName

FROM Employee E

INNER JOIN Department D

ON E.DeptID = D.DeptID;

```
mysql> SELECT *
-> FROM Employee, Department
-> WHERE Employee.DeptID = Department.DeptID;
```

EmpID	EmpName	Salary	DeptID	DeptID	DeptName
101	Arun	55000.00	D1	D1	IT
103	Rahul	65000.00	D1	D1	IT
102	Priya	45000.00	D2	D2	CSE
105	Vijay	70000.00	D2	D2	CSE
104	Sneha	40000.00	D3	D3	ECE

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT E.EmpID,
->         E.EmpName,
->         D.DeptName
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Arun	IT
103	Rahul	IT
102	Priya	CSE
105	Vijay	CSE
104	Sneha	ECE

```
5 rows in set (0.00 sec)
```

Explanation: Uses INNER JOIN to avoid unnecessary duplicate results.

Question 2

Debug the missing correlation in a subquery.

Incorrect Query

SELECT EmpName

FROM Employee

WHERE Salary >

(

SELECT AVG(Salary)

FROM Employee

);

Correct Query

SELECT EmpName

FROM Employee E

WHERE Salary >

(

SELECT AVG(Salary)

FROM Employee

WHERE DeptID = E.DeptID

);

```
+-----+
| EmpName |
+-----+
| Rahul   |
| Vijay   |
+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT EmpName
-> FROM Employee E
-> WHERE Salary >
-> (
-> SELECT AVG(Salary)
-> FROM Employee
-> WHERE DeptID = E.DeptID
-> );
+-----+
| EmpName |
+-----+
| Rahul   |
| Vijay   |
+-----+
2 rows in set (0.00 sec)
```

Explanation: The subquery is now correlated using DeptID.

Question 3

Debug the SQL View.

Incorrect Query

CREATE VIEW EmployeeView AS

*SELECT **

FROM Employee;

Correct Query

CREATE VIEW EmployeeView AS

SELECT EmpID,

EmpName,

Salary,

DeptID

FROM Employee;

```
mysql> SHOW FULL TABLES;
+-----+-----+
| Tables_in_companydb | Table_type |
+-----+-----+
| bankaccount          | BASE TABLE |
| department           | BASE TABLE |
| employee             | BASE TABLE |
| employee1            | BASE TABLE |
| employeeproject      | BASE TABLE |
| employeeview         | VIEW        |
| it_employees         | VIEW        |
| productstock         | BASE TABLE |
| project              | BASE TABLE |
| sales                | BASE TABLE |
| salestrigger         | BASE TABLE |
| student              | BASE TABLE |
+-----+-----+
12 rows in set (0.00 sec)
```

*Explanation: Avoid using SELECT * in views. Specify only the required columns.*

Question 4

Debug the Relational Algebra Expression.

Incorrect

Employee ⋈ Project

Correct

Employee ⋈ Employee.ProjectID = Project.ProjectID Project

Explanation: Specify the join condition to perform the correct natural/equi-join.

Question 5

Fix Referential Integrity.

Incorrect

*INSERT INTO Employee
VALUES(101,'Arun','D5');*

Correct

*INSERT INTO Department
VALUES('D5','IT');*

*INSERT INTO Employee
VALUES(101,'Arun','D5');*

```
mysql> SHOW FULL TABLES;
+-----+-----+
| Tables_in_companydb | Table_type |
+-----+-----+
| bankaccount          | BASE TABLE |
| department           | BASE TABLE |
| employee             | BASE TABLE |
| employee1            | BASE TABLE |
| employeeproject      | BASE TABLE |
| employeeview         | VIEW        |
| it_employees         | VIEW        |
| productstock        | BASE TABLE |
| project              | BASE TABLE |
| sales                | BASE TABLE |
| salestrigger         | BASE TABLE |
| student              | BASE TABLE |
+-----+-----+
12 rows in set (0.00 sec)
```

Explanation: The parent (Department) record must exist before inserting the child (Employee) record.

Question 6

Rewrite Nested Query using JOIN.

Inefficient Query

```
SELECT EmpName
FROM Employee
WHERE DeptID IN
(
SELECT DeptID
FROM Department
);
```

Optimized Query

```
SELECT E.EmpName,
       D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```

EmpName
Arun
Priya
Rahul
Sneha
Vijay

-> ON E.DeptID = D.DeptID;	
EmpName	DeptName
Arun	IT
Priya	CSE
Rahul	IT
Sneha	ECE
Vijay	CSE

Explanation: INNER JOIN is more efficient than a nested subquery in this case.

Question 7

Debug GROUP BY Query.

Incorrect

SELECT DeptID,

EmpName,

AVG(Salary)

FROM Employee

GROUP BY DeptID;

Correct

SELECT DeptID,

AVG(Salary) AS AverageSalary

FROM Employee

GROUP BY DeptID;

```
mysql> SELECT DeptID,
->      EmpName,
->      AVG(Salary)
-> FROM Employee
-> GROUP BY DeptID;
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'companydb.Employee.EmpName' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
mysql> SELECT DeptID,
->      AVG(Salary) AS AverageSalary
-> FROM Employee
-> GROUP BY DeptID;
```

DeptID	AverageSalary
D1	60000.000000
D2	57500.000000
D3	40000.000000

Explanation: Every selected non-aggregated column must appear in the GROUP BY clause.

Question 8

Fix UPDATE Statement.

Incorrect

UPDATE Employee

SET Salary = Salary + 5000;

Correct

UPDATE Employee

SET Salary = Salary + 5000

WHERE EmpID = 101;

```
mysql> UPDATE Employee
-> SET Salary = Salary + 5000;
Query OK, 5 rows affected (0.01 sec)
Rows matched: 5  Changed: 5  Warnings: 0

mysql> UPDATE Employee
-> SET Salary = Salary + 5000
-> WHERE EmpID = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Explanation: The WHERE clause prevents updating every row.

Question 9

Optimize Multi-table Query.

Inefficient Query

```
SELECT *  
FROM Employee  
WHERE DeptID IN  
(  
SELECT DeptID  
FROM Department  
);
```

Optimized Query

```
SELECT E.EmpName,  
       D.DeptName  
FROM Employee E  
INNER JOIN Department D  
ON E.DeptID = D.DeptID;
```

Recommended Indexes

```
CREATE INDEX idx_emp_dept  
ON Employee(DeptID);
```

```
CREATE INDEX idx_dept_id  
ON Department(DeptID);
```

EmpID	EmpName	Salary	DeptID
101	Arun	65000.00	D1
102	Priya	50000.00	D2
103	Rahul	70000.00	D1
104	Sneha	45000.00	D3
105	Vijay	75000.00	D2

EmpName	DeptName
Arun	IT
Priya	CSE
Rahul	IT
Sneha	ECE
Vijay	CSE

Explanation: Indexes improve join performance and reduce query execution time.

Question 10

Debug Relational Calculus.

Incorrect

$\{E \mid \forall D(\text{Employee}(E) \wedge \text{Department}(D) \wedge E.\text{DeptID}=D.\text{DeptID})\}$

Correct

$\{E \mid \text{Employee}(E) \wedge \exists D(\text{Department}(D) \wedge E.\text{DeptID}=D.\text{DeptID})\}$

Explanation: $\exists$ (there exists) is appropriate here. Using $\forall$ (for all) incorrectly makes the condition too restrictive, often resulting in an empty result set.

Code of Conduct:

I _____K.VIJAYALAKSHMI(192511189)_____ (Name / Reg No) certify that this submission is my

original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:K.VIJAYALAKSHMI